BrandLens

Brand governance with receipts · final report, Rounds 1-3 · November 12, 2026

BrandLens reviews marketing content against a brand kit and answers, for every finding, the only two questions a brand team actually argues about: which clause of the guidelines does this break, and where in the document does it break it. Every finding cites its clause and its page. Every decision carries a reviewer, a timestamp and a hash. Nothing about the judgement is a black box, and nothing about the record can be quietly edited.

Open it

Live site	https://site-a3e60bfeac664aac8b66164e0943ddf4.freebuff.page
Source	https://github.com/ishmeettt/brandlens
Demo	judge@brandlens.demo / BrandLens-Judges-2026, or "Enter demo workspace" on the sign-in page
Stack	Next.js 16 (App Router) · React 19 · TypeScript · Supabase (Postgres + RLS) · Cloudflare Workers

---

1. The problem, in one paragraph

Brand teams ship decks and documents, and brand review is still a copy-paste ritual: someone pastes a slide into a document, a brand lead reads it, and the answer comes back as "this feels off" with no clause, no page and no record of who approved what. The cost is not the review itself, it is that every exception becomes permanent because nobody can find the decision again, and every rule becomes unarguable because nobody knows how often it is right. BrandLens makes the review mechanical where it can be, cited where it matters, and auditable after the fact.

2. Every feature, one per line

All of the following work on the live link today.

- Email and password sign-in with creator, reviewer and manager roles, enforced by Postgres row-level security rather than by the interface.
- A public demo workspace with a seeded brand kit, four submissions and eleven findings, generated by the engine itself.
- Paste-text submission for a quick check.
- PDF submission parsed server-side, page by page, inside the Worker.
- Scanned PDFs routed to OCR on the server instead of being rejected.
- DOCX submission with paragraph anchors that carry the nearest heading.
- Multi-document batch submission: several files reviewed as one set with one combined verdict.
- Page-anchored citations: a finding on page 14 cites p.14 and the sentence it came from.
- Document-prefixed anchors inside a set, so a finding names the deck it came from.
- A deterministic rule engine covering terminology, claims, contrast, palette and specificity.
- Every finding carries its brand-kit clause, source passage, locator and suggestion.
- Unreadable uploads rejected with a 422 and a readable reason instead of an empty submission.

- Per-finding reviewer decisions, accept or dismiss, each with an optional note.
- Submission verdicts: approve or request changes, with accepted, dismissed and pending counts.
- A brand kit editor for colours, fonts, logo rules, terminology and guideline clauses.
- Scoped, owned, expiring exemptions that the engine honours while in date and lapses on its own afterwards.
- A policy clock that derives a deadline from the worst finding on a submission (critical 1 day, high 3, medium 5, low 7).
- An explicit due date that overrides the derived one, and an assigned reviewer for every submission.
- Queue filters: Needs review, Mine, Overdue.
- An admin dashboard with workspace totals, findings by severity, and per-rule precision and false-positive rate with a thin-sample marker.
- An audit record per submission exported as a PDF and as JSON from the same sealed pack, with a SHA-256 digest.
- Engine reconciliation inside the audit record: which findings the current engine still reproduces, and what it would raise that was never recorded.
- A decision log where every accept, dismiss and approval is written to an append-only table and hash-chained.
- Chain verification in the browser, with the page stating on screen that the log was read from the write-once feed.
- Reviewer and timestamp attribution on every decision in the feed and in the audit record.

3. Technical architecture

Next.js 16 App Router with React 19 and TypeScript, server routes for anything that touches a file, Supabase as the system of record, and Cloudflare Workers as the runtime through the OpenNext adapter.

Client	React 19 pages under src/app, supabase-js with the publishable key only, no service key in the bundle.
Intake	POST /api/extract parses PDF (pdf.js legacy build) and DOCX (mammoth) server-side, with tesseract OCR for scanned PDFs and a browser fallback parser.
Engine	src/lib/rules.ts, a pure deterministic module; same input, same findings, no model in the loop.
Pipeline	src/lib/review-pipeline.ts runs the engine, applies exemption state and severity accumulation, and optionally layers an AI pass.
Governance	governance-ops.ts for the clock and SLA, exemptions stored in brand_kits.guidelines, governance-insights.ts for per-rule precision.
Record	audit-pack.ts builds one canonical pack; audit-pack-pdf.ts renders it through pdf.ts, a dependency-free A4 writer.
Ledger	audit_events, append-only by trigger, each row carrying the hash of the row before it; audit-events.ts appends and verifies.
Hosting	Cloudflare Workers with the parsers bundled and an isolated R2 framework cache; Supabase hosted Postgres with RLS.

Data flow. A submission enters through /submit. Files go to /api/extract, which returns one anchored segment per page or paragraph; the text is stored with marker lines ([[p.4]], [[para 12 - Pricing]], [[Autumn deck - p.4]]). Anchors live inside the text rather than in a parallel index, which means no schema change was needed to ship them, anything that reads the stored text still gets prose, and an anchor recorded today still cites the same page a year from now. The engine sees the text with markers stripped; each finding recovers its locator by mapping its evidence token back to the segment it fell inside.

The data model is deliberately small: profiles (role, organisation), brand_kits (guidelines JSON including exemptions), content_submissions (anchored text plus a document map, owner and due date), findings (severity, clause, locator, rule_id, review state, reviewer and timestamp), and audit_events (append-only decision feed). Authorization is expressed once, in RLS policies, so the interface cannot be the place where permissions are decided.

The hardest problem was making PDF parsing genuinely server-side on Workers. OpenNext first traced the parsers into node_modules and could not symlink them on Windows, and a Worker has no node_modules at runtime, so both parsers are now bundled into the Worker. pdf.js then needed its worker registered in-process because a Worker cannot load one from a file URL. The second hardest was making the hash chain verify: PostgREST returns +00:00 timestamps while the hashes were computed over Z, so every valid chain read as tampered until timestamps were normalised on both write and verify. The append-only trigger then blocked its own on-delete-set-null foreign key, which needed a migration that permits exactly that one update and nothing else.

4. The three improvements we committed to at the end of Round 2

Graded against the Round 2 submission, so each is answered as committed versus delivered, including the parts that did not land.

Improvement 1 - Ingest the content teams actually ship

Committed	PDF and DOCX text extraction, page-anchored citations, batch review of a whole set; documents parsed server-side, text mapped back to page and paragraph, a finding on page 14 citing page 14 and the exact sentence, several documents reviewed as one set with a combined verdict.
Delivered	All of it, at the pipeline level. Server-side PDF and DOCX parsing, page anchors for PDF, paragraph-plus-heading anchors for DOCX, document-prefixed anchors in a set, locators carried into the review and the audit record, and batch submissions reviewed as one set with one verdict.

Beyond the brief, because the format demanded it: scanned PDFs are detected and routed to OCR instead of being rejected, and a file that cannot be read at all returns 422 with a reason rather than becoming an empty submission. The review screen states whether the text was parsed on the server or in the browser, so the claim is visible rather than implied.

Evidence: a real 7-page PDF uploaded to the deployed site parsed server-side, returned 200 with 7 pages and 12,371 characters, and the form reported "parsed on the server"; an image-only PDF produced 1 page, 15 words, 110 characters through the OCR path.

Caveat: DOCX is built and anchored but has not been exercised against a real Word file on the deployment, because the corpus has no real .docx. PDF is the format verified end to end.

Improvement 2 - Governance operations: ownership, deadlines, expiring exemptions, exportabl

Committed	An assigned reviewer and due date per submission, an SLA view of what is late, scoped exemptions that the engine honours and then expires, and one record exporting as a PDF plus an append-only feed for legal with reviewer, timestamp, finding and clause.
Delivered	All of it. Ownership and due dates persist, the policy clock drives Mine and Overdue views, exemptions are scoped, owned, dated and lapse without anything being deleted, decisions are written to an append-only hash-chained feed with reviewer and timestamp, and the audit record exports as PDF and JSON from one sealed pack.

The PDF export is the piece the plan named that the product did not have: the record previously produced JSON and a browser print dialog. It now renders a real A4 PDF through a dependency-free writer, from the same sealed pack as the JSON, so the readable copy cannot say something the machine copy does not. Both carry the digest, the anchors, the verdict, every finding with its clause, the engine reconciliation and the exemptions that applied.

Evidence: owner and due date saved and read back on the live database; a reviewer decision recorded with reviewed_by and reviewed_at; the decision log showing "Chain verified" read from the write-once feed with three entries from two different reviewers; Mine listing the judge's finding and Overdue empty because the deadline is two days out; the PDF export producing a 7,354-byte application/pdf on the live URL.

Improvement 3 - Trust and measurement: labelled corpus, per-rule precision, tamper-evident lo

Committed	A fixed labelled corpus per rule family built from real submissions, run on every change and model upgrade, precision and false-positive rate per rule on Admin, a hash-chained audit log, and SSO/SAML for workspaces that need it.
Delivered	The corpus, the gate, the per-rule precision, and the tamper-evident log. SSO/SAML was not delivered.

The corpus is a labelled set of cases per rule family in the repository, and a replay harness runs the real engine against it, reporting precision and recall per rule and exiting non-zero if a labelled case is missed or an unlabelled finding appears. That gate is what turns a model upgrade into a measurable event: the same command that proves the rules today has to pass after the change. Admin surfaces the same measurement where a brand lead will look, mapping decisions back to the exact rule through the stored rule_id, with a thin-sample marker and a separate unattributed bucket so the number cannot flatter itself.

The audit log is hash-chained end to end. Decisions are written to a table a trigger refuses to update or delete, each row carrying the previous hash; the page recomputes the chain in the browser and reports it verified or broken. Editing a past decision breaks every hash after it, and the guarantee does not rest on the client, because the database enforces it independently.

SSO/SAML is the one item in this improvement that was not built. Supabase's SAML support is a paid-plan feature and needs an identity provider with metadata and certificates that only the workspace owner can supply; a fake SSO screen would have been worse than none. Sign-in stays on email and password, the gap is written into docs/round-3.md, and the migration path is a Supabase SAML connection plus one provider entry, with no change to the shape of the application code.

Evidence: eval:rules reports 100% precision and recall across the labelled corpus and fails the build otherwise; test:governance covers the clock, per-rule precision, exemptions, the audit pack and the chain; the live decision log verified three chained entries written by two reviewers.

5. Other work after Round 2

- Server-side parsing on Workers: parsers bundled instead of traced, pdf.js worker registered in-process, and the review screen made honest about where parsing happened.
- Scanned-PDF OCR path with a six-page ceiling and a browser fallback.
- An append-only decision feed, first written from the client and then moved to a write-once table with a trigger.
- Three database migrations to close defects the features exposed: owner and due date columns with audit events; a policy allowing the foreign key's own nulling without weakening the write-once rule; a read policy so reviewers can read the feed head they append to.
- Timestamp normalisation in the hash chain, without which every valid chain verified as broken.
- Demo seeding rebuilt around the real engine, with a dry-run plan and a reconciliation self-check, so seeded audit records reproduce their own findings.
- Owner, due date and team passwords added to the seed, so the demo shows a live SLA and reviewers can actually sign in.
- React effect defects fixed across review, admin, audit, brand kit and the shell: decisions no longer wiped by a state reset, no stuck spinners, no double fetches.
- An image object-URL leak on the review and submit previews fixed.
- DOCX anchor naming and changes_requested counting fixed.
- Generated build output excluded from lint: 28,923 problems to 0, without deleting tracked files.
- A dependency-free PDF writer added for the audit export, and this report generated with it.
- Dead configuration removed (a stray tsconfig, an unused supabase-config.js), and the documentation corrected where it described migrations as unapplied.

6. Testing

What was tested, how, and what is still not perfect.

Rule accuracy	eval:rules replays the labelled corpus through the real engine, reports precision and recall per rule, and gates the build. Current: corpus matches the engine exactly.
Governance	test:governance exercises the policy clock, per-rule precision, exemptions, the audit pack and the hash chain.
Database policies	verify:supabase runs 16 checks against the hosted database: the full creator-to-reviewer flow, anonymous read refusal, creator privilege limits, and cleanup. 16 of 16 pass.
Seed integrity	seed:demo --dry-run prints the plan without writing and self-checks reconciliation: 4 submissions, 11 findings, 11 of 11 reproduced, 0 engine-only, 0 unmatched.
Build	tsc clean, eslint clean, next build clean across 12 routes.
Live, manual	A real PDF upload and parse, a scanned PDF through OCR, owner and due date persistence, a reviewer decision attributed and chained, the decision log verifying three entries, Mine and Overdue filters, the PDF export download, and a route sweep where all eight pages returned 200 and /api/extract returned 405 to GET and 422 to an empty POST.

Fixed during testing	A forked audit chain, a chain that read as broken due to timestamp format, an append-only trigger that made a reviewed submission undeletable, seeded findings that did not reconcile, a review screen that wiped decisions, an object-URL leak, and the lint flood from generated output.
Still imperfect	No automated browser end-to-end suite; DOCX unverified against a real Word file; two simultaneous decisions could still fork the chain; the AI pass is off on the deployment; SSO is absent.

7. Rounds 1 to 3, in order

Round 1 - the core loop. Submit content, run a deterministic rule engine against the brand kit, get findings that each cite a clause, and have a reviewer accept or dismiss every one before approval. Brand kit editor, submission queue, review workspace, Supabase schema with row-level security, and a seeded demo that runs without credentials.

Round 2 - the governance layer. A policy clock that derives a deadline from severity, scoped and expiring exemptions, a decision log, an audit pack with a SHA-256 digest, per-rule precision on Admin, an optional AI judgement pass layered over the deterministic engine, and a labelled corpus with a replay harness.

Round 3 - the three improvements above, plus the defects they exposed: server-side extraction with anchors and OCR, batch review, ownership and SLA, expiring exemptions honoured by the engine, a PDF and JSON audit pack, an append-only chained feed, and measurement that gates a rule change. The round also produced three migrations and the timestamp fix that made the chain verifiable.

8. Where it goes next

Ordered by what blocks a paying team soonest, with the audience each item serves.

1. Server-side chain	Compute the chain inside Postgres under an advisory lock so concurrent decisions cannot fork it. Blocks: any team with more than one reviewer. Effort: 1 to 2 days. This is the only correctness gap left in the record.
2. DOCX and PPTX, verified	Put a real deck and a real Word document in the corpus, parse speaker notes and layout order, and gate both in eval:rules. Audience: marketing and design teams, who ship decks first. Effort: about a week.
3. Assignment board	A workload view for managers, reassignment, and a daily overdue digest by email or Slack. Audience: brand operations leads. Effort: about a week. The data is already modelled.
4. Intake integrations	A Figma plugin and a Google Drive folder watch, so review starts where the work is made rather than at an upload form. Audience: designers. Effort: 2 to 3 weeks, plus an app review for each platform.
5. Rule authoring with evidence	Let a brand manager write a rule in the kit editor and immediately see it against the corpus and the last 90 days of submissions before enabling it. Audience: brand managers tuning noisy rules. Effort: 2 weeks.
6. SSO/SAML and SCIM	The Round 3 gap, unlocked by a Supabase plan and an identity provider. Audience: enterprise legal and IT. Effort: about 3 weeks, mostly configuration and a second tenant to test against.
7. Legal-grade export	Signed PDFs with a timestamp authority and a verification page that a third party can check without our app. Audience: legal teams. Effort: 2 weeks.

8. Precision over time	Trend lines per rule and an A/B view for a model upgrade, so the dashboard answers "is this getting better" as well as "is this right". Audience: whoever owns brand quality. Effort: 1 to 2 weeks.

9. Why this is the winning project

- It makes a claim and proves it in the product, not in a slide. A finding cites a clause, a decision cites a reviewer, a record carries a digest, and the accuracy figure comes from a labelled corpus that gates the build.
- The trust features are systems, not screens. The database refuses edits to the decision feed, the chain is recomputed and reported, and precision is per rule with thin samples marked instead of a flattering average.
- It handles the work as it exists. A 40-page deck and a launch document go in as one set and come out as findings with page numbers, which is the difference between a demo and something a team would use on a Monday.
- It is honest about its edges. SSO, DOCX verification and the concurrent-append race are written down with reasons in the product, in docs/round-3.md and in this report, which is the same discipline the product asks of its users.
- It runs. The public link works without credentials, the migrations are applied to a real database with real policies, the parsers execute inside the Worker, and the demo data reconciles with its own audit.
- The engineering holds up under inspection: deterministic engine, RLS as the only authorization boundary, one canonical pack behind two export formats, and a failure mode for every input rather than a silent empty submission.

Live site	https://site-a3e60bfeac664aac8b66164e0943ddf4.freebuff.page
Repository	https://github.com/ishmeettt/brandlens
Demo sign-in	judge@brandlens.demo · BrandLens-Judges-2026

BrandLens · brand governance with receipts · this report was generated by the same PDF writer the product uses for its audit export (scripts/make-report.mjs).